

Dycard: A Lightweight and Deployable Cardinality Refinement System with Online Updates

Jiatao Hu
Tsinghua University
hhjtt110@gmail.com

Yongwei Wu
Tsinghua University
wuyw@tsinghua.edu.cn

Abstract

In recent years, numerous studies have explored the use of machine learning techniques for accurate cardinality estimation to enhance query optimization. However, existing learning-based cardinality estimation methods often face challenges in maintaining robustness and ease of integration, limiting their practicality in real-world database systems. In this paper, we propose Dycard, a practical and deployable solution tailored for real-world database systems. Unlike many existing machine learning-based approaches, Dycard fully leverages the results of native optimizers, eliminating the need for pre-training and enabling seamless integration for immediate deployment in production environments.

Dycard operates as an auxiliary refinement system that incrementally adjusts cardinality estimates in real time through a streaming training and inference framework. As queries are executed, Dycard continuously updates its lightweight model while providing online refinement to the native estimator. By employing an attention-based query encoding, it effectively captures complex join-crossing correlations and offers an interpretable view into inter-column dependencies. We integrated Dycard into PostgreSQL and found that it significantly reduces cardinality estimation errors and improves query performance with minimal latency overhead.

PVLDB Reference Format:

Jiatao Hu and Yongwei Wu. Dycard: A Lightweight and Deployable Cardinality Refinement System with Online Updates. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/hujiatao0/Dycard>.

1 Introduction

The query optimizer plays an important role in the performance of database systems. By selecting efficient execution plans for SQL queries, the optimizer significantly affects the overall system throughput and latency. With the rapid advances in machine learning, recent years have seen a surge of interest in applying learning-based techniques to query optimization [25, 66, 73]. Some studies have proposed building end-to-end learned optimizers that predict execution plans directly from queries [4, 7, 36, 63]. Others

have focused on learning specific components of the optimizer pipeline, such as improving the accuracy of cardinality estimation [22, 24, 28, 42–44, 58, 69], or training cost models that better reflect real-world execution behavior [17, 30, 34, 38, 46, 49, 61].

Among these components, accurate cardinality estimation has been shown to have a more critical impact on plan quality than the design of cost models [26]. As a result, improving cardinality estimation has become a particularly active and important research direction in learned query optimization. To effectively apply cardinality estimation improvements in production, several key challenges must be addressed:

Accurate cardinality estimation Traditional estimators rely on assumptions such as uniformity and independence, which rarely hold in real-world datasets. As a result, optimizers often fail to capture correlations across tables and columns. When a query involves joins between highly correlated tables or multiple predicates on correlated attributes, the resulting cardinality estimates can be significantly inaccurate. Moreover, such estimation errors tend to propagate and amplify as the number of joins and predicates increases, frequently leading to severely suboptimal execution plans. Recent research has explored two main directions for learning-based cardinality estimation. *Data-driven* approaches aim to learn the joint distribution of the underlying data, treating queries as structured access patterns over this learned distribution [18, 64, 75]. In contrast, *query-driven* methods learn a direct mapping from query features to cardinality values by training models on query workloads [22, 48, 54]. All of these methods aim to capture the complex correlations inherent in relational data through learned models, thereby improving the accuracy of cardinality estimation.

Minimal training overhead A critical requirement for integrating learned cardinality estimators into production environments is minimizing training overhead while avoiding cold-start degradation. Most existing approaches, whether query-driven or data-driven, rely on non-negligible pretraining phases before reaching usable accuracy. Query-driven approaches typically require extensive historical workloads to collect subquery-cardinality pairs for supervised learning. The data acquisition phase alone may take several hours or more. In contrast, data-driven methods tend to exhibit shorter preparation times, but their model construction cost increases with the size of database, and they often incur significant storage and computational overhead. For complex or large-scale databases, certain learning-based techniques may even become intractable due to resources constraints [21, 56].

Adaptivity to dynamic data drift Data distribution drift caused by insertions and deletions is common in production environments. For purely query-driven approaches, which learn a direct mapping

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

from query structure to cardinality, this poses a significant challenge. These models are typically trained on static snapshots of the database and cannot adapt once the underlying data changes. As a result, substantial data modifications often need costly resampling and model retraining procedures. Data-driven methods offer partial relief through incremental updates to model parameters, which can reduce reconstruction time under moderate data changes. However, these approaches still face deployment challenges due to their lack of explicit handling of query patterns during training, often leading to higher inference costs compared to query-driven models.

Recent work has explored various strategies to address the challenge. ALECE [27] adopts a hybrid design that decouples SQL queries from the underlying data distribution and applies attention-based mechanisms to jointly model structural and statistical patterns, improving robustness under dynamic workloads. LPCE [54] maintains a query-driven design by learning two models: one for baseline cardinality estimation and another for progressively refining estimates and reconstructing bad plans as more queries are observed. However, both methods remain dependent on access to large volumes of original training data and incur non-trivial training overhead, which limits their applicability in production.

Controllability of negative optimization effects A common integration strategy adopted by most learned cardinality estimation methods is full replacement—substituting the native estimator with a machine learning model that is either fully trained or partially pre-trained. While such models may achieve improved accuracy under controlled benchmarks, they inherently introduce uncertainty due to the stochastic nature of learning algorithms and their limited interpretability. This uncertainty becomes particularly problematic in production environments characterized by large, heterogeneous datasets and diverse, complex query patterns.

In such settings, even a small fraction of misestimated queries may lead to severely degraded plans with high execution cost. More importantly, database administrators and users often lack sufficient visibility into the behavior of learned components, making it difficult to diagnose errors or trust the model’s decisions [56]. Without mechanisms for fallback or constraint enforcement, such systems risk causing non-negligible negative performance impacts that are hard to mitigate once deployed.

Lightweight and integrable deployment To the best of our knowledge, no existing learned cardinality estimation method has been fully integrated as a usable module within a real DBMS. As a core component of the query optimizer, the effectiveness of cardinality estimator is measured not only by its estimation quality, but also by how well it integrates with the system’s architecture and whether it can provide optimization services with minimal runtime and engineering overhead in production.

Achieving this requires a holistic evaluation of the database system’s behavior with the learned module in place. The integration should be lightweight in terms of resource overhead, ensuring that the system remains stable under realistic workloads. Moreover, it is essential to clearly articulate the trade-offs associated with such integration—including inference latency, memory footprint, and fallback behavior—so that database administrators can make informed decisions regarding adoption and configuration.

Motivated by the above challenges, we propose Dycard, a cardinality refinement system that operates alongside the native cardinality estimator rather than replacing it. Dycard provides refined cardinality estimates during query planning, acting as an auxiliary mechanism that enhances the native optimizer’s decisions.

Our integration strategy follows a similar philosophy to prior learning-based optimization systems such as Bao [35], Lero [72], and LEON [6]. Bao applies query hints and feedback to optimize plan choices, while Lero and LEON learn to rank candidate plans using execution feedback. Unlike Bao, Lero, and LEON—which all perform end-to-end query plan optimization—Dycard focuses solely on refining cardinality estimates, without intervening in the plan enumeration or selection process. This design is motivated by several considerations: (1) Among the three core components of a query optimizer—cardinality estimation, cost modeling, and plan enumeration—cardinality estimation remains a primary bottleneck. Significant estimation errors can lead to catastrophic plan choices, whereas the room for improvement in cost models is comparatively limited [26]. Moreover, real-world DBMSs already include mature and well-engineered plan enumeration strategies, such as genetic algorithms [3, 23], rule-based enumeration in the Cascades framework [13], and pruned dynamic programming [45]. With reasonably accurate cardinality estimates, the search space—even if imperfect—tends to yield competitive plans. As demonstrated in Section 5.2, we show that dynamic refinement of cardinalities alone can achieve performance on par with end-to-end learning approaches. (2) Many end-to-end optimizers are themselves dependent on cardinality estimates during candidate plan generation. For example, Bao directly uses PostgreSQL’s internal cardinality estimator, and Lero adjusts estimated cardinalities as a knob to generate different candidate plans. In both cases, large estimation errors can degrade the quality of the candidate plan pool, thereby impairing the optimizer’s ability to choose an efficient plan. (3) Focusing exclusively on cardinality estimation improves modularity and extensibility. This design makes Dycard easier to integrate with existing systems and orthogonal to other optimization techniques that benefit from improved estimates. It provides a flexible and adaptable foundation for future enhancements without requiring changes to the optimizer’s cost model or plan search logic.

From the system’s internal design, Dycard adopts a streaming framework for both training and inference, which prevents query processing from being blocked by model updates or sample collection. The system continuously refines its model by learning from executed queries and incrementally updating its parameters with minimal latency. In addition, we propose a novel query encoding strategy to improve the mapping from subqueries to estimated cardinalities. Inspired by recent advances in natural language processing, our method perfectly formulates query encoding as an attention-based [52] sentence encoding problem. This approach not only enables more effective extraction of query patterns but also offers an interpretable view into learned join-crossing correlations, thereby mitigating the black-box nature commonly associated with deep learning models.

In summary, we make the following contributions in this work:

- We introduce Dycard, a lightweight and auxiliary cardinality refinement system that can be integrated into the PostgreSQL. Dycard

progressively adjusts erroneous cardinality estimates based on executed queries and can outperform end-to-end optimizers under both static and dynamic data distributions. To the best of our knowledge, it is the first learned cardinality estimation method designed to be tightly coupled with the kernel of a real DBMS and used in actual query processing.

- Dycard adopts a streaming architecture with decoupled training and inference modules. We design an efficient synchronization mechanism that enables real-time cardinality refinement while supporting efficient model updates.
- We propose a novel attention-based query encoding scheme that captures query features more effectively and provides a theoretically grounded and interpretable view of join-crossing correlations, facilitating model visualization and monitoring.
- We design a new optimization objective and loss function that eliminates the need for pretraining, enabling zero-sample cold-start operation without degrading query performance.
- Dycard supports runtime toggling via a simple configuration switch, allowing user to enable or disable the system at any time. This built-in fallback mechanism mitigates potential negative impacts from learning uncertainty, making Dycard practically deployable in production environments.

2 System Overview

Dycard is a lightweight cardinality estimation refinement system designed to enhance existing database estimator through progressive adaptation. The architecture of Dycard is shown in Figure 1.

When enabled, Dycard interacts with each query at two key stages of its processing. In the planning phase, Dycard analyzes each subquery by extracting relevant features, such as the involved tables and predicates, along with the estimated cardinality. These are then fed into the model to generate a corrected cardinality, which is synchronously returned to the DBMS kernel for query optimization. In the execution phase, once the query’s physical execution plan has been fully executed, Dycard gathers query features and actual cardinalities from the subnodes of the physical plan. These are treated as samples and added to the sample queue, creating a dataset for future model training.

Dycard provides real-time, online cardinality refinement for DBMS, continuously improving the model by training on samples collected from executed queries. This iterative process demands low inference latency and efficient synchronous training. To meet these requirements, Dycard decouples its training and inference modules, enabling parallel processing through a multi-process framework. Communication among the modules is streamlined using a cross-process queue and shared memory. Then we will provide a brief explanation of some details in these components.

Model Management Dycard maintains separate model instances for inference and training, each managed by its corresponding module. Both models share the same parameter structure, allowing synchronization of updates between modules. This design enables Dycard to maintain stable and consistent inference behavior during query optimization.

To support model input and sample construction, we apply an attention-based query featurization that encodes each subquery into a variable-length matrix, where each row represents a feature

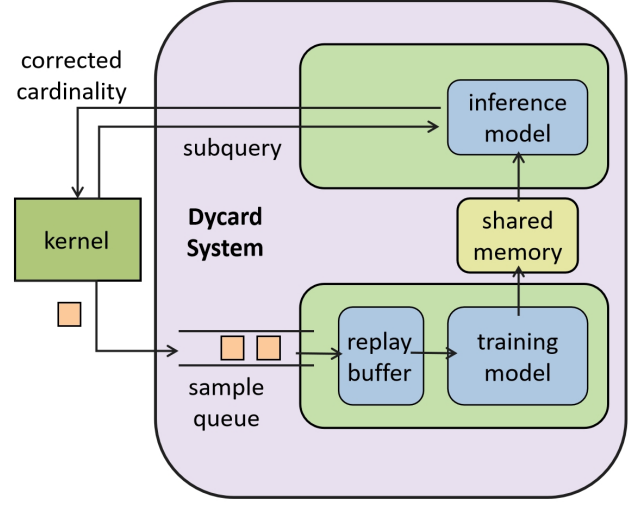


Figure 1: The architecture of Dycard

vector corresponding to a column involved in the query. This representation captures structural elements such as predicates and join relationships, and serves as input to models. The details of the encoding design will be discussed in Section 3.

Additionally, Dycard provides model persistence interfaces that support saving and loading model states. These interfaces enable offline model migration, support pretraining workflows, and improve deployment flexibility by allowing models to be transferred or restored across environments.

Model Training Dycard’s training process aligns with traditional query-driven cardinality estimation methods, treating the task as a supervised regression problem. Unlike approaches that directly predict cardinalities from queries, we focus on learning the difference between the actual cardinality and the native DBMS estimate for each sample, allowing the model to adjust rather than replace the original estimate. To ensure this refinement works well in real time, training operates as an online streaming process, running in parallel with query execution without causing interference.

Since obtaining training samples involves executing each query, which can be resource-intensive, it’s crucial to make the most of every sample. We incorporate a replay buffer mechanism inspired by reinforcement learning[32]. By enabling the model to train on samples across multiple iterations, this design significantly improves stability and speeds up convergence during training.

Module Communication To enable real-time cardinality refinement, Dycard relies on two key communication mechanisms: a sample queue for collecting training samples from the DBMS kernel, and a shared memory region for synchronizing model parameters between the training and inference modules.

The sample queue serves as an interface between the DBMS kernel and Dycard’s internal training pipeline. After query execution, the kernel extracts subquery-level cardinality information and asynchronously pushes it into the sample queue. This decoupling allows the DBMS engine to remain non-blocking, as the training

module can deal with samples independently without interfering with ongoing query processing.

Parameter synchronization between the training and inference models is handled through a shared memory mechanism. After each training iteration, updated model parameters are written to shared memory. The inference module accesses this shared state before each inference cycle, ensuring that it always operates on the latest available model. To maintain consistency, a locking mechanism prevents simultaneous reads and writes to the shared memory space, allowing both modules to run concurrently without conflict.

3 Query Featurization

Query featurization is important to all query-driven cardinality problems. Our approach leverages the structural similarity between query encoding and sentence representation in NLP, allowing seamless use of attention mechanism for query feature extraction. It also provides an interpretable view of join-crossing correlations learning, enabling analysis of intermediate outcomes and reducing the lack of transparency typical in deep learning models.

3.1 Classic Attention Mechanism

In natural language processing (NLP), a common approach to textual representation involves decomposing a sentence into a sequence of tokens through tokenization. Each token serves as a fundamental unit and is mapped to a corresponding embedding vector, typically derived from a pre-trained embedding space. These vectors are then concatenated to form a sentence-level representation matrix. Formally, we denote the sentence as a matrix $X = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_L]^\top \in \mathbb{R}^{L \times d}$, where each row $\mathbf{x}_i$ represents the embedding vector of the i -th token in the sentence and L denotes the number of tokens in the input sequence.

Transformer model has emerged as a highly effective solution in the context of large-scale language modeling. At the core of the Transformer lies the attention mechanism, which has proven instrumental in capturing contextual relationships between tokens. Due to its simplicity and expressive power, attention has become a foundational component in many modern language models and has been extensively studied in recent years.

The core computation of the attention mechanism begins by projecting the input sentence matrix X into three distinct spaces through learnable linear transformations. Specifically, the matrices W_Q , W_K , and W_V are applied to obtain the query (Q), key (K), and value (V) representations:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

where $W_Q, W_K \in \mathbb{R}^{d \times d_{qk}}$ and $W_V \in \mathbb{R}^{d \times d_v}$ are trainable weight matrices and d is the dimensionality of the embedding space. Also, in the computation of attention, W_Q and W_K share the same dimensionality, while W_V may have a different dimension. We denote these dimensions as d_{qk} for W_Q and W_K , and d_v for W_V , respectively.

Once Q , K , and V are obtained, the output of the attention module is computed as a weighted sum of the value vectors. The weights are determined by the scaled dot-product attention mechanism, formulated as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V. \quad (1)$$

This operation captures the pairwise interactions between tokens by computing the similarity between queries and keys, which are then normalized through the softmax function to produce attention weights. These weights are subsequently used to aggregate the value vectors, enabling the model to dynamically focus on relevant parts of the input sequence.

Specifically, The matrix multiplication $QK^\top \in \mathbb{R}^{L \times L}$ computes the dot product between each query and key pair. After applying the scaling factor $\frac{1}{\sqrt{d}}$ and the softmax function, we obtain the attention weight matrix $A \in \mathbb{R}^{L \times L}$, which determines how much attention each token pays to every other token in the sequence.

Each element $A_{i,j}$ in the matrix represents the normalized attention weight from the i -th token (as a query) to the j -th token (as a key), defined as:

$$A_{i,j} = \frac{\exp\left(\frac{q_i \cdot k_j}{\sqrt{d}}\right)}{\sum_{j'=1}^L \exp\left(\frac{q_i \cdot k_{j'}}{\sqrt{d}}\right)}. \quad (2)$$

where q_i and k_j are the i -th row of Q and j -th row of K , respectively. Intuitively, $A_{i,j}$ quantifies the similarity between token i and token j , and reflects the degree to which token i attends to token j during the encoding process. The resulting $L \times L$ matrix captures the inter-token dependencies within the sentence, allowing the model to dynamically reweight the influence of different tokens. This mechanism enables the attention module to learn contextual relationships by highlighting which parts of the sequence are more relevant to each token's representation. A notable advantage of this property lies in its ability to facilitate the visual analysis of inter-token relationships—an aspect that we have fully exploited in the encoding design presented in Section 3.2.

3.2 Query Encoding

3.2.1 column encoding The core idea of our approach is to treat each column involved in a query—either as part of the predicates or the join conditions—as tokens, similar to those in natural language sentences. All such columns are encoded into vectors and concatenated to form a matrix representing the query, analogous to the token matrix in NLP-based sentence encodings. The encoding process is illustrated in Figure 2.

Specifically, we initialize separate embedding vectors for both tables and columns based on the schema of the underlying database. These embeddings are precomputed and indexed by name, allowing for efficient lookup during the encoding process. The initialization process is lightweight, typically requiring only a few seconds, and it needs to be performed only once. The resulting embeddings are saved and reloaded alongside the model state during training.

Each column's final representation is constructed by concatenating three components: the embedding of the associated table, the embedding of the column itself, and an encoding of the filter condition (if applicable). For predicate columns, the filter condition is decomposed into two parts: the operator (e.g., $>$, $=$, $<$) and the filter value. The operator is encoded using a simple one-hot vector, while the filter value is normalized to a value between 0 and 1 based on the statistical metadata of the column, such as min/max ranges, to facilitate effective model training. For join columns that do not involve any filter conditions, we append a default vector of

Query: select count(*) from comments as c, posthistory as ph where c.userid = ph.userid and c.creationdate <= 1410412138 and c.score = 1 and ph.creationdate <= 1409902660;

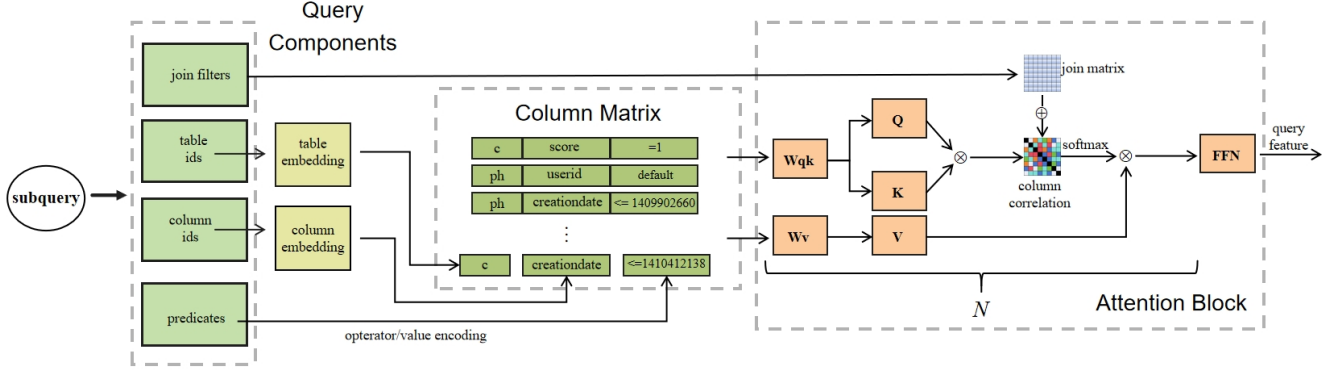


Figure 2: Query Featurization of Dycard

fixed length in place of the filter condition encoding, ensuring a consistent input format across different types of columns.

This column-oriented encoding scheme serves to isolate the fundamental units that influence cardinality estimation. Specifically, the cardinality of a query is determined solely by the columns it involves—namely predicate columns and join columns—as well as the join relationships among join columns. It is independent of the physical operators used in the query plan at that node. Each subquery within a query can thus be viewed as a subset of the overall set of columns and their corresponding join relationships. Our encoding strategy focuses on abstracting the essential features that fundamentally impact cardinality estimation. This abstraction not only aligns with the underlying semantics of the task but also establishes a direct analogy to sentence encoding in NLP. A key difference, however, is that while sentence encoding requires positional information to preserve the order of tokens—typically via position encodings—the columns in a query are inherently unordered; their semantic contribution is independent of listing order. As such, we omit the use of position encoding in our model, which simplifies the architecture while preserving its ability to model column-level dependencies. This design naturally paves the way for applying attention mechanisms in an interpretable manner.

3.2.2 Attention with join matrix Traditional cardinality estimators in systems like PostgreSQL are based on the independence assumption, which treats predicates and joins as statistically independent events. While this assumption performs reasonably well for single-table queries, it fails to capture correlations across columns from different tables—commonly referred to as join-crossing correlations. As a result, the inaccuracy of cardinality estimates often leads to suboptimal and inefficient query execution plans. To address this challenge, we introduce a novel attention mechanism augmented with a *join matrix*. This design allows the model to explicitly encode and learn the relationships between columns involved in join operations across different tables. This targeted attention improves the model’s ability to capture join-crossing correlations, ultimately enhancing the precision of cardinality refinement in complex query

		R.id	S.id	R.a>0	
R.a>0	R.id	1.0	0.8	0.8	0
	S.id	0.8	1.0	0.1	0
	R.id	0.8	0.1	1.0	0
	S.id	0	0	0	0
	R.id	0	0	0	0

Figure 3: Join Matrix with Batch Padding

scenarios. For each input query, we first transform it into a column sequence matrix $X \in \mathbb{R}^{L \times d}$ following the encoding process described in Section 3.2.1, where L denotes the number of encoded columns and d is the fixed embedding dimension for each column. We then apply three trainable linear transformations: $W_q \in \mathbb{R}^{d \times d_{qk}}$, $W_k \in \mathbb{R}^{d \times d_{qk}}$, and $W_v \in \mathbb{R}^{d \times d_v}$, which project the input matrix X into the query, key, and value spaces.

At this point, the dot product $QK^T \in \mathbb{R}^{L \times L}$ can be interpreted as a correlation matrix between every pair of columns in the query. Each entry in this matrix quantifies the structural correlation between a pair of encoded columns. While in NLP applications the resulting attention scores typically represent how much attention one token pays to another, our setting interprets the matrix QK^T differently. Here, the matrix serves as a symmetric representation of pairwise correlations among columns, reflecting their potential semantic or structural interactions—particularly those that may arise from join conditions or shared participation in predicates. Since the notion of column correlation is inherently symmetric. We enforce this property by using a shared projection matrix $W_{qk} \in \mathbb{R}^{d \times d_{qk}}$ for both queries and keys, setting $Q = XW_{qk}$ and $K = XW_{qk}$. This design ensures that the resulting matrix QK^T remains symmetric by construction, which better aligns with the semantics of relational

data. Moreover, it reduces the number of trainable parameters, contributing to a more compact and efficient model. This formulation preserves the expressive capacity of the attention mechanism while aligning it with the relational characteristics of SQL queries.

Building on the correlation matrix computed via attention, we further introduce a *join matrix* to encode explicit join relationships within a query. Prior studies have proposed various ways to capture such structural information. For example, Neo [36] employs an adjacency matrix to represent pairwise join relationships among columns and flattens the upper triangular part of this symmetric matrix into a one-dimensional vector for encoding. MSCN [22], on the other hand, extracts neural representations of tables, columns, and joins separately and concatenates them as input features. While these approaches incorporate some aspects of join structure, they often fall short in fully exploiting the pairwise nature of join interactions and may incur additional architectural complexity.

In contrast, we adopt a more direct and interpretable method. We manually construct a *join matrix* that captures pairwise join relationships among columns and add it element-wise to the attention-derived column correlation matrix. The construction of our *join matrix* $M_{\text{join}} \in \mathbb{R}^{L \times L}$ is defined as follows:

$$M(i, j) = \begin{cases} 1.0, & \text{if } c_i = c_j \\ 0.8, & \text{if } c_i \bowtie_{\text{eq}} c_j \text{ or } T(c_i, c_j) \\ 0.4, & \text{if } c_i \bowtie_{\text{neq}} c_j \\ 0.6, & \text{if } c_i \bowtie_{\text{gt}} c_j \quad (M(j, i) = -0.6) \\ 0.1, & \text{otherwise} \end{cases} \quad (3)$$

This formulation encodes join semantics as a structured correlation matrix over columns. Diagonal entries, representing self-correlations, are assigned the maximum value of 1.0, reflecting their perfect internal consistency. Pairs of columns involved in equality joins, or columns belonging to the same table (denoted as $T(c_i, c_j)$), are assigned a relatively high weight of 0.8, indicating strong association. Inequality joins involving greater-than or less-than conditions are encoded with asymmetric weights (e.g., 0.6 for $M(i, j)$, -0.6 for $M(j, i)$) to reflect directional dependencies. Non-equality joins are assigned a lower correlation score of 0.4. For all other column pairs with no direct join or relational connection, a minimal default value of 0.1 is used to indicate weak or no correlation. Figure 3 provides a concrete example of the designed *join matrix*. Notably, we assign the same correlation weight to pairs of columns within the same table as we do to columns explicitly joined via equality conditions. This design choice is motivated by the observation that intra-table columns often share implicit relationships through the primary key or indexing structure. For instance, columns such as $R.\text{id}$ and $R.a$ are inherently correlated due to their common association with the primary key $R.\text{id}$, which acts as a join anchor within the table. Therefore, we assign the same correlation score (0.8) to $R.\text{id} \sim R.a$ as to $R.\text{id} \sim S.\text{id}$, where the latter is defined by an explicit equality join condition $R.\text{id} = S.\text{id}$. This alignment ensures semantic consistency between intra-table and inter-table join representations in the encoded matrix.

In practice, training is performed in batches, where the number of encoded columns varies across queries within a batch. As a result, the corresponding *join matrices* differ in size. To enable efficient batch processing, we adopt a simple yet effective padding strategy:

each *join matrix* is zero-padded to match the maximum column number within the batch. This ensures uniform dimensionality for matrix addition and attention computation across the batch, without introducing spurious correlations.

To integrate structural priors with learned correlations, we add the static join matrix $M_{\text{join}} \in \mathbb{R}^{L \times L}$ to the attention score matrix $QK^\top / \sqrt{d}$, resulting in a fused correlation matrix prior to softmax normalization:

$$A = \text{softmax} \left(\frac{QK^\top}{\sqrt{d}} + M_{\text{join}} \right), \quad (4)$$

where $A \in \mathbb{R}^{L \times L}$ denotes the final attention weight matrix. Each entry $A_{i,j}$ lies in the range $[0, 1]$, representing the learned relevance between column c_i and column c_j , now guided by both data-driven attention and explicit structural knowledge.

The formulation enables the model to learn flexible column dependencies while being anchored by known join relationships, resulting in a structurally-aware attention map. Furthermore, the resulting matrix A serves not only as input to subsequent layers, but also as an interpretable intermediate artifact. It offers valuable insights into how the model perceives join-crossing correlations—a key challenge in classical cardinality estimation. We present detailed visualization and interpretability analysis of these attention weights in Section 5.4.

4 Dynamic Cardinality Refinement

Dycard is designed as an auxiliary cardinality refinement system that works in tandem with the kernel of DBMS to enhance query optimization. Rather than replacing the native estimator, Dycard refines its output through dynamic feedback. The system operates via three communication interfaces with the kernel and contributes to both the planning and execution phases of query processing.

During the planning phase, Dycard receives subquery information along with the corresponding cardinality estimates produced by the native estimator of DBMS. These inputs are fed into the inference model, which returns a correction delta. This delta is used to refine the original estimate before the final physical plan is generated. Once the query has been fully executed, the kernel invokes the `add_training_sample` interface to submit structural and cardinality information for each subquery into Dycard’s cross-process sample queue, thereby populating the training set for model updates in the future.

Dycard adopts a streaming training and inference framework that allows near real-time updates and online refinement with minimal resource consumption. This avoids the overhead and staleness issues often associated with centralized offline training, ensuring that the model evolves continuously with the workload. We fully integrate Dycard into the open-source PostgreSQL engine, and it can be toggled at runtime using the familiar configuration interface like Bao [35] and Lero [72], e.g., `SET enable_dycard = true/false;`. When disabled, PostgreSQL falls back to its native cardinality estimation and bypasses Dycard’s sampling and training pipeline. This provides database administrators with flexibility in deployment and allows Dycard to coexist with other orthogonal optimization techniques that benefit from improved cardinality estimates.

4.1 Model Training

The training process of Dycard is initiated intermittently rather than continuously. The training module periodically (usually few seconds) monitors the sample queue to assess the number of accumulated training samples. When the number of samples reaches a predetermined threshold, a training process is triggered.

During each training process, all samples in the replay buffer are iterated over for N rounds, where N is a tunable parameter that balances training precision against overfitting and computational overhead. This iterative approach ensures that the model is sufficiently refined by leveraging both recent and representative historical query information. Consequently, Dycard enables rapid model iteration and convergence, allowing it to quickly incorporate feedback from newly executed queries.

Learning Cardinality Residuals Dycard follows a query-driven learning paradigm similar to existing approaches, in which the goal is to learn a mapping from query features to cardinality estimates. However, rather than directly predicting the true cardinality, Dycard learns to estimate the residual between the true value and the estimate produced by the native database optimizer.

Formally, let x denote the feature representation of a query and $\hat{c}_n$ the corresponding cardinality estimate produced by the native estimator. The model learns a function f_θ such that:

$$f_\theta(x, \hat{c}_n) \approx c_t - \hat{c}_n \quad (5)$$

where c_t is the ground-truth cardinality observed after query execution. This residual learning formulation allows the model to operate as a lightweight correction layer, refining estimates without replacing the native estimator. It also stabilizes the learning process by focusing on smaller output magnitudes, which are typically easier to regress than absolute cardinality values.

This residual-based formulation also enables a particularly elegant initialization strategy. At the start of training, the parameters of the last fully connected layer are initialized from a distribution with a small mean, ensuring that the model’s output is close to zero across different inputs. This means that Dycard initially applies negligible adjustments to the native cardinality estimates, effectively depending on the native estimator in the absence of experience.

This cold-start-safe behavior is particularly important for production deployment. Unlike many learning-based optimizers that require warm-up periods or offline pretraining to achieve reasonable performance, Dycard maintains stable and reliable behavior even in zero-sample settings—a property that is rare among existing learned query optimization approaches.

Loss Function In query-driven cardinality estimation, the task is typically framed as a supervised regression problem that maps query features to cardinality values. However, a key challenge arises from the extreme variability in cardinality magnitudes—often spanning several orders of magnitude—which makes direct supervision on absolute counts problematic. Training on raw cardinality values tends to induce large gradient fluctuations, thereby increasing the risk of unstable optimization.

To address this, a common practice is to apply a logarithmic transformation to both the predicted and true values, effectively compressing the value range and stabilizing the gradient scale. The

standard log-space mean squared error (MSE) loss is defined as:

$$MSE_Loss = \frac{1}{N} \sum_{i=1}^N (\log(\hat{y}_i) - \log(y_i))^2 \quad (6)$$

where $\hat{y}_i$ denotes the predicted cardinality and y_i is the ground-truth value for the i -th sample.

This loss function implicitly measures the ratio between predicted and true cardinalities, aligning closely with the definition of *Q-error*, a commonly used metric in cardinality estimation. The *Q-error* for a single prediction is defined as:

$$Q\text{-error} = \max\left(\frac{\hat{y}_i}{y_i}, \frac{y_i}{\hat{y}_i}\right) \quad (7)$$

where $\hat{y}_i$ and y_i denote the predicted and true cardinalities, respectively. This formulation penalizes relative errors symmetrically in log space, making log-MSE a natural proxy for optimizing *Q-error*.

However, recent studies [15, 41] have pointed out that *Q-error* does not always correlate well with real-world query performance, particularly in terms of execution latency. For example, consider two queries with true and predicted cardinalities of (10, 40) and (10,000, 40,000), respectively. Both yield a *Q-error* of 4, yet the performance impact of the estimation error in the latter case is typically much more severe due to the significantly larger data volume and resulting execution plan cost.

This observation suggests that while *Q-error* provides a useful relative measure of estimation quality, it may overlook absolute deviations that are critical for latency-sensitive planning.

To address this limitation, we design a new loss function that incorporates both the relative and absolute magnitude of the target cardinalities. The modified loss is defined as:

$$L = \frac{1}{N} \sum_{i=1}^N \lambda^{\log(y_i)} (\log(\hat{y}_i) - \log(y_i))^2 \quad (8)$$

where λ is a tunable hyperparameter. Unlike the standard log-MSE loss, this formulation applies an exponential weight based on the logarithm of the true cardinality. Intuitively, it amplifies the gradient contribution of samples with higher cardinality values, where absolute estimation errors are more likely to lead to severe performance degradation in execution. This weighted loss encourages the model to focus more aggressively on reducing high-impact errors while still maintaining the relative error structure captured by the log-space formulation. In effect, it steers the model toward prioritized convergence on samples whose misestimation would most adversely affect query performance. Moreover, since Dycard applies cardinality refinement to all subqueries within a query plan, it inherently mitigates the compounding effect of small estimation errors that can propagate and amplify across join operations—a challenge highlighted by Moerkotte et al. [39].

4.2 Streaming framework

Dycard adopts an online learning framework [2, 11] that incrementally updates the model using continuously collected query samples. Unlike conventional batch learning approaches—which often require large volumes of data and periodic, centralized training—Dycard’s streaming-based strategy offers improved resource efficiency by spreading the computational load over time. As we

demonstrate in Section 5, mini-batch training in this setting imposes minimal overhead and does not interfere with the execution of live queries. Beyond its efficiency benefits, the online learning paradigm also offers natural adaptability to workload drift. As data distributions evolve, Dycard continuously incorporates new observations into the training process, allowing the model to remain up-to-date without requiring full retraining or manual intervention.

To ensure that cardinality refinement remains non-blocking, Dycard decouples model training from inference. After a query is executed, it is decomposed into multiple subqueries, each producing a labeled training sample. These samples are pushed to a cross-process sample queue. Once a configurable mini-batch threshold is reached, a batch of samples is pulled into a replay buffer, which triggers a new training process. To manage training efficiency and maintain model quality over time, Dycard employs a configurable replay buffer with a FIFO eviction policy. The design is motivated by the dynamic nature of query samples in an online refinement setting. Since Dycard adjusts cardinalities at the subquery level, and native estimators propagate subquery estimates upward during query planning, repeated queries may produce different subquery training samples as the refined estimates evolve. As a result, the training set naturally accumulates both newer, more accurate samples and older, potentially outdated ones.

The FIFO-based replay buffer helps mitigate the adverse effects of stale samples by gradually phasing them out. This design also imposes an upper bound on the number of samples used in each training cycle, preventing excessive training overhead even when the sample stream is uneven or bursts occur.

Upon completion of each training cycle, the training module writes updated parameters to shared memory. The inference module checks a synchronization flag before each inference task, and if updates are detected, it refreshes its parameters accordingly. A locking mechanism ensures safe concurrent access, preventing race conditions when serving multiple queries in parallel.

The entire process is implemented in a fully asynchronous, non-blocking manner. This design guarantees that inference and training operate independently and efficiently, ensuring real-time cardinality refinement without compromising system responsiveness.

5 EXPERIMENTAL EVALUATION

The goal of our experimental study is to evaluate whether Dycard can serve as a practical and deployable cardinality refinement system in real-world database environments. To this end, we design a comprehensive set of experiments that assess both the effectiveness and robustness of Dycard from multiple perspectives.

In Section 5.2, we evaluate Dycard’s performance in terms of Q-error and query execution latency, comparing it against native and learning-based baselines. Section 5.3 investigates the potential negative optimization effects introduced by learning-based methods, analyzing Dycard’s stability. In Section 5.4, we conduct ablation studies and interpretability analysis on our attention-based query encoding design to understand the capture of join-crossing correlations. Finally, we examine the cost overhead introduced by Dycard during query optimization, including memory footprint, training latency, and inference time in Section 5.5.

5.1 Experimental Setup

Dataset and workload preparation We evaluate Dycard on two widely-used relational datasets: IMDB and STATS, which differ in schema complexity and data distribution characteristics.

IMDB is a classic movie-related dataset containing 21 relational tables, originally used in the Join Order Benchmark (JOB)[26]. STATS is a more recent dataset extracted from the Stats Stack Exchange platform, consisting of 8 tables with highly skewed and correlated attributes across users, posts, and activities. Compared to IMDB, STATS presents a more challenging environment for learning-based cardinality estimation due to its intricate data distribution.

For each dataset, we generate two separate workloads: a *warm-up workload* and a *test workload*.

The warm-up workload consists of 1,000 randomly generated queries created by sampling join paths from the schema graph and attaching different selection predicates. This workload is used to simulate zero-shot deployment scenarios and analyze Dycard’s ability to refine cardinality estimates. And the test workload contains 800 queries generated from existing benchmark query templates (i.e., JOB for IMDB and STATS-CEB[15] for STATS). Each query in the test workload is generated by randomly selecting a query template from the benchmark, extracting its join structure, and regenerating different predicates. It ensures that all test queries are distinct from those in the warm-up workload. This workload is used for all baseline comparisons and dynamic workload evaluations.

To evaluate Dycard’s adaptability under dynamic workloads, we simulate data distribution drift throughout the test process. Specifically, we divide the test workload into sequential batches, each containing 20 queries. Between each batch, we inject 5,000 randomly generated data modification statements—consisting of insertions and deletions—distributed across all tables in the dataset. These updates introduce controlled shifts in data distribution and simulate the behavior of a live database system under evolving workloads. Following each update phase, we invoke the ANALYZE command on all affected tables to refresh the internal database statistics. This ensures that both the native optimizer and Dycard operate on up-to-date information. For other baseline comparisons, we apply the same procedure.

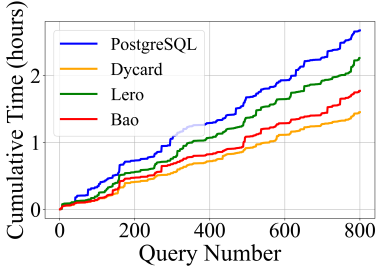
Baselines We do not compare Dycard against fully trained cardinality estimation models, as many of these approaches are difficult to integrate into real DBMS kernels and adopt fundamentally different optimization paradigms. Instead, we focus on baselines that are both comparable in terms of incremental learning behavior and practically deployable. Specifically, we select Bao [35] and Lero [72] as our primary baselines. Both systems are open-source end-to-end optimizers which can be natively integrated with PostgreSQL. In addition, we include the PostgreSQL as a baseline to reflect the performance of the native query optimizer without any learning-based enhancement.

Bao is a learned query optimizer that uses reinforcement learning to improve the plan selection process. It interacts with PostgreSQL through a hint-based interface and gradually optimizes plan choices based on query feedback.

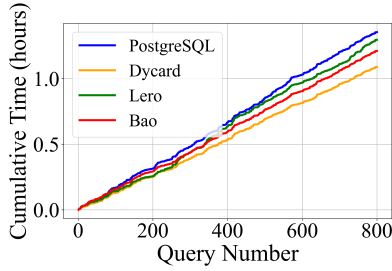
Lero is a learning-to-rank optimizer that improves native plan selection by training a pairwise classifier to compare candidate plans.

Table 1: Q-error Comparison Summary

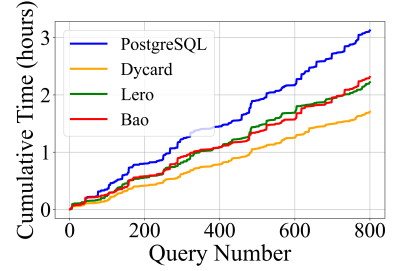
DATASETS	Query	50%			90%			95%			99%		
		PG	Dycard	↓	PG	Dycard	↓	PG	Dycard	↓	PG	Dycard	↓
IMDB	1-200	6.00	5.90	1.67%	$1.07 \cdot 10^2$	$1.03 \cdot 10^2$	3.74%	$1.93 \cdot 10^2$	$1.92 \cdot 10^2$	0.52%	$2.71 \cdot 10^3$	$2.61 \cdot 10^3$	3.69%
	201-400	$1.12 \cdot 10^1$	8.00	28.57%	$1.32 \cdot 10^2$	$1.04 \cdot 10^2$	21.21%	$1.88 \cdot 10^3$	$9.35 \cdot 10^2$	50.27%	$3.45 \cdot 10^3$	$2.58 \cdot 10^3$	25.22%
	401-600	$1.42 \cdot 10^1$	5.67	60.07%	$1.60 \cdot 10^2$	$4.79 \cdot 10^1$	70.06%	$5.19 \cdot 10^2$	$1.53 \cdot 10^2$	70.52%	$3.45 \cdot 10^3$	$1.31 \cdot 10^3$	62.03%
	601-800	8.49	2.72	67.96%	$1.60 \cdot 10^2$	$1.88 \cdot 10^1$	88.25%	$5.43 \cdot 10^2$	$3.52 \cdot 10^1$	93.52%	$3.39 \cdot 10^3$	$2.56 \cdot 10^2$	92.45%
	801-1000	8.46	2.68	68.32%	$1.66 \cdot 10^2$	$1.77 \cdot 10^1$	89.34%	$5.38 \cdot 10^2$	$3.41 \cdot 10^1$	93.66%	$3.76 \cdot 10^3$	$2.37 \cdot 10^2$	93.70%
STATS	1-200	$1.22 \cdot 10^1$	$1.56 \cdot 10^1$	-27.87%	$6.83 \cdot 10^4$	$4.52 \cdot 10^4$	33.82%	$7.33 \cdot 10^5$	$5.02 \cdot 10^5$	31.51%	$6.41 \cdot 10^7$	$1.63 \cdot 10^7$	74.57%
	201-400	7.64	$1.11 \cdot 10^1$	-45.29%	$2.06 \cdot 10^3$	$2.30 \cdot 10^3$	-11.65%	$3.72 \cdot 10^5$	$2.45 \cdot 10^4$	93.41%	$2.19 \cdot 10^7$	$1.06 \cdot 10^6$	95.16%
	401-600	$1.23 \cdot 10^1$	9.40	23.58%	$8.88 \cdot 10^3$	$5.36 \cdot 10^2$	93.96%	$1.55 \cdot 10^5$	$4.70 \cdot 10^3$	96.97%	$1.22 \cdot 10^8$	$2.45 \cdot 10^4$	99.98%
	601-800	$1.23 \cdot 10^1$	6.06	50.73%	$1.19 \cdot 10^5$	$4.86 \cdot 10^2$	99.59%	$1.87 \cdot 10^6$	$2.17 \cdot 10^3$	99.88%	$6.75 \cdot 10^7$	$2.22 \cdot 10^4$	99.97%
	801-1000	$1.56 \cdot 10^1$	6.02	61.41%	$1.09 \cdot 10^4$	$1.71 \cdot 10^2$	98.43%	$1.00 \cdot 10^5$	$3.98 \cdot 10^2$	99.60%	$6.41 \cdot 10^7$	$2.06 \cdot 10^4$	99.97%



(a) STATS Test(Static Workload)



(b) IMDB Test(Static Workload)



(c) STATS Test(Dynamic Workload)

Figure 4: Latency Comparison in Different Workload

It integrates with PostgreSQL non-intrusively and continuously refines its ranking model based on query execution feedback.

Environment and Parameter Setup Our experiment environment is in cloud and the machine is equipped with 128GB of memory and an Intel® Xeon® Ice Lake processor (2.9GHz), running Ubuntu 22.04 LTS. The implementation uses Python 3.11 and PyTorch 2.1.0 as the deep learning framework.

The Dycard model adopts a deep attention-based architecture with 10 attention layers for query encoding and refinement. To manage sample throughput and prevent overload from rapid short-query executions, we set the sample queue capacity to 3,000. The replay buffer is limited to 1,000 entries. For the weighted loss described in Section 4.1, we empirically determine that setting the hyperparameter $\lambda = 2$ yields the best performance trade-off. Each training process is configured to run only a single epoch per batch. We observe that higher iteration counts can lead to overfitting, particularly in earlier training stages where sample diversity is limited.

5.2 Performance Across Data Distributions

We evaluate Dycard’s performance using two key metrics: Q-error and end-to-end query latency. These metrics reflect the system’s ability to progressively refine cardinality estimates and improve actual query plans under real workload conditions.

Table 1 summarizes the Q-error distributions on the IMDB and STATS datasets, starting from zero-sample initialization. We evaluate it in warm-up workload and the queries are grouped into batches of 200. The estimation quality of Dycard is evaluated after each batch to assess its progressive refinement capability.

We show that Dycard converges rapidly and consistently reduces estimation error across the query workload. Notably, Dycard demonstrates particularly strong improvements for high Q-error outlier queries. On IMDB, even with fewer than 800 warm-up queries, Dycard nearly reduces the 90th percentile Q-error by over 90%. On the more complex STATS dataset, Dycard achieves similar or better improvements in fewer than 600 queries, where the 95% and 99% Q-errors are reduced by up to two orders of magnitude in many cases. The improvement is especially pronounced for queries with severe baseline misestimates, where Dycard’s refinement is most impactful.

To evaluate Dycard’s practical impact on overall query performance, we compare its end-to-end query latency with that of other optimization approaches. Since Bao relies on prior experience and Lero benefits from partial pretraining, we apply a consistent warm-up procedure for fairness. Specifically, all methods are prewarmed on the same 1,000-query warm-up workload, after which we measure cumulative execution time on the test set of 800 queries. The results are shown in Figure 4.

As illustrated in Figures 4a and 4b, Dycard consistently outperforms both Bao and Lero across the STATS and IMDB datasets. The

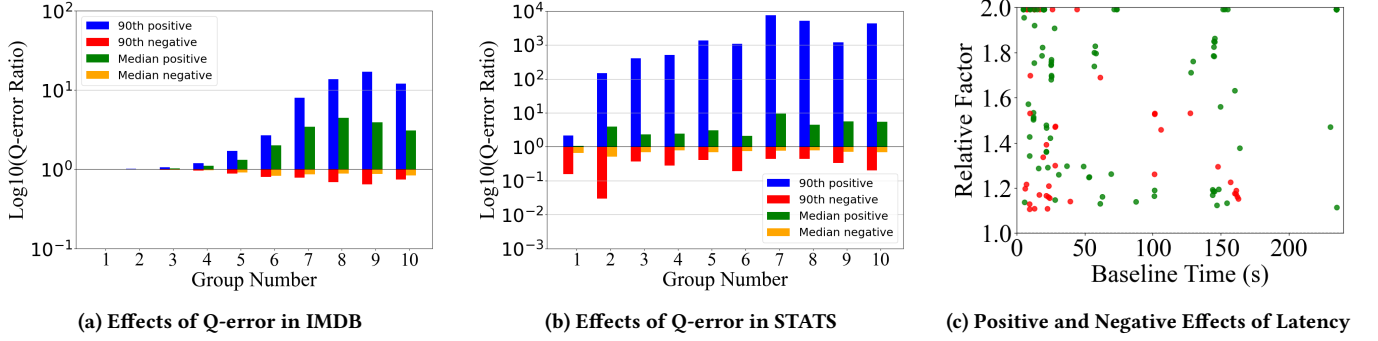


Figure 5: Negative Effects

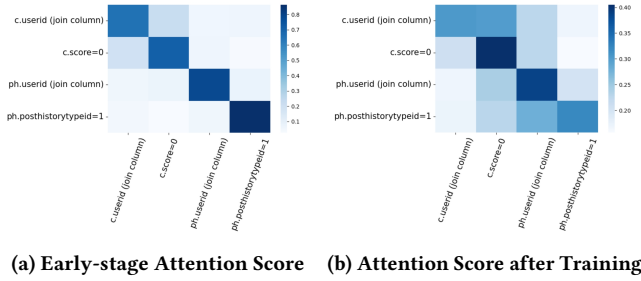


Figure 6: Visualization of Attention Scores

Table 2: Query Completion Count after 2 Hours

Method	Queries	Improvement (%)
PostgreSQL	647	—
Neo	692	6.96
MSCN	855	32.15
Ours	1126	74.03

improvement is especially notable on STATS, which features more complex distributions and larger baseline estimation errors. There, Dycard reduces total execution time by nearly half compared to native PostgreSQL, demonstrating its ability to eliminate inefficient plans caused by poor cardinality estimates. On IMDB, where estimation errors are generally smaller, Dycard still shows competitive performance, though the margin of improvement is narrower.

These results reinforce that Dycard, by refining only the cardinality estimation module, can achieve performance on par with or better than fully end-to-end learned optimizers—while providing easier integration and better extensibility.

In Figure 4c, we further evaluate Dycard under dynamic workloads on STATS dataset. As expected, Dycard adapts effectively to shifting data distributions by leveraging the native optimizer’s ability to incorporate up-to-date statistics through ANALYZE, while simultaneously applying learned residual corrections online. This hybrid capability allows Dycard to remain robust under data drift and maintain the best overall latency among all evaluated systems.

5.3 Negative Optimization Effects

We also investigate the potential for negative optimization—cases where learned corrections may degrade rather than enhance execution quality. This analysis is especially important for deep learning-based systems deployed in production, where stability and robustness are critical concerns.

Figures 5a and 5b visualize the Q-error impact of Dycard on the IMDB and STATS datasets, respectively, during progressive warm-up. We divide the query workload into 10 consecutive groups (each containing 100 queries) and record the positive and negative optimization effects in each group. For each group, we compute the ratio between Dycard’s estimate and the native estimate, and report both median and 90th percentile improvements (in blue/green) and degradations (in red/yellow) on a logarithmic scale.

On the IMDB dataset, we observe a clear trend of increasing positive optimization over time, while the magnitude of negative effects remains consistently low throughout the process. And On the STATS dataset, Dycard initially exhibits slightly higher negative variance during early-stage learning, but these effects are rapidly suppressed as more feedback is incorporated. Meanwhile, the positive optimization grows and dominates in later stages.

These results demonstrate that Dycard is not only effective in reducing large estimation errors, but also capable of doing so without introducing much instability.

To further assess Dycard’s behavior in terms of execution time, we analyze its latency-level optimization effects in Figure 5c. The horizontal axis represents the baseline query latency measured on native PostgreSQL, while the vertical axis shows the relative time ratio after Dycard’s intervention.

Each dot represents a single query. Green dots indicate positive optimization cases, where Dycard successfully reduced execution time; their vertical value is defined as the ratio between baseline latency and Dycard latency. Red dots represent negative optimization cases, where latency increased; their vertical value reflects the inverse ratio (Dycard latency divided by baseline latency).

The plot reveals a clear trend: queries with longer execution times tend to benefit from Dycard’s optimization, with few or no negative effects observed in this high-latency region. In contrast, negative optimization is primarily concentrated among short-running queries (e.g., <50ms), where even if cardinality estimates

Table 3: Comparison of Planning and Execution Time for Different Numbers of Joins (Unit: ms)

Joins	Planning Percentiles (Dycard)				Planning Avg			Execution Avg		
	P50	P90	P95	P99	PostgreSQL	Dycard	↑	PostgreSQL	Dycard	↓
2	7.10	8.90	9.10	9.40	0.70	7.40	6.7	18467.1	17029.5	1437.6
3	25.30	29.30	31.30	36.60	1.10	24.80	23.7	642.1	535.5	106.6
4	67.00	78.00	80.60	90.70	1.60	65.90	64.3	10572.7	4761.7	5811.0
5	156.10	182.40	191.90	198.60	3.20	151.00	147.8	29169.4	12524.5	16644.9
6	254.90	289.20	295.50	329.40	3.60	258.10	254.5	8995.0	5307.5	3687.5
7	498.50	535.40	539.80	552.40	6.50	500.80	494.3	19949.1	17959.1	1990.0

Table 4: Trade-off between CPU and Memory Usage

Resource	Mode	Avg	Max	P90	P95	P99	Std
CPU	PostgreSQL	6.43	8.18	6.56	6.62	7.07	0.17
	Dycard	7.33	65.36	6.75	11.26	38.03	4.96
Memory	PostgreSQL	4.92	4.97	4.95	4.95	4.97	0.02
	Dycard	5.54	5.89	5.56	5.57	5.80	0.06

are suboptimal, the resulting increase in execution time remains marginal due to the low absolute cost of these queries.

This behavior is a direct consequence of our loss design discussed in Section 4.1, where absolute cardinality magnitude influences the weighting of each training sample. As a result, subqueries with larger expected impact—typically those with higher cardinalities and cost—receive greater attention during training, enabling more substantial improvements. Short queries, while sometimes imperfectly optimized, incur negligible overhead due to their already minimal latency.

Moreover, the overall distribution in Figure 5c shows that positive optimization cases significantly outnumber negative ones, both in quantity and impact. This further demonstrates Dycard’s robustness in real workloads and its ability to improve end-to-end performance without sacrificing stability.

5.4 Encoding Experiments

Ablation Study on Encoding We conduct an ablation study to evaluate the effectiveness of our attention-based query encoding design in comparison to two representative alternatives from prior work: Neo [36] and MSCN [22].

The first baseline follows the encoding strategy proposed in Neo, which encodes join relationships using the upper-triangular portion of a join graph’s adjacency matrix. The matrix is flattened into a one-dimensional vector to serve as the encoding of join structure. For predicate encoding, we adopt histogram-based encoding, which captures selectivity information through binned feature vectors—among the several schemes discussed in Neo, including one-hot and row vector encoding. The second baseline implements the MSCN approach, which separately encodes the sets of tables, predicates, and join conditions using independent neural networks.

These representations are aggregated using average pooling and concatenated to form the final query vector.

We implemented both methods and replaced Dycard’s internal encoding module with each baseline in turn. Under a fixed warm-up setting of 1,000 queries, we ran all variants for two hours and recorded the total number of completed queries. As shown in Table 2, our encoding method enables Dycard to process significantly more queries within the same time window—exceeding PostgreSQL by over 70% and outperforming MSCN and Neo by wide margins. This demonstrates that our attention-based encoding is more effective at capturing structural and statistical query features, leading to better cardinality refinement and lower query latency.

Interpretability of Attention-Based Encoding One key advantage of our encoding design is that it naturally lends itself to interpretability analysis, by drawing a direct analogy with attention-based sentence encoders in natural language processing (NLP). By treating each predicate or join column as a “token” in a query, our approach enables us to visualize how the model captures inter-column relationships via attention scores.

Figure 6 illustrates this effect using a representative query from the STATS-CEB workload:

```
SELECT COUNT(*) FROM comments AS c, posthistory
AS ph WHERE c.userid = ph.userid AND c.score
= 0 AND ph.posthistorytypeid = 1;
```

This query involves two join columns and two selection predicates, resulting in a 4×4 attention matrix when encoded without padding.

Figure 6a shows the attention weights after early-stage training. The model initially focuses on self-correlations, yielding a near-diagonal matrix where each column mostly attends to itself. However, after several rounds of query-driven refinement, the attention map in Figure 6b reveals a more nuanced understanding: darker off-diagonal entries emerge, highlighting learned correlations between columns from different tables.

For example, the strong attention weight between `c.userid` and `ph.userid` reflects the explicit join condition in the query, while the interaction between `c.score=0` and `ph.posthistorytypeid=1` may indicate that the model has captured higher-order statistical dependencies. These learned attention patterns influence downstream encoding and ultimately improve cardinality estimation.

5.5 Optimization Cost

As Dycard introduces learned cardinality refinement during query planning, it inevitably incurs additional computational overhead, particularly in the planning phase. This overhead stems from the need to apply the inference model to each subquery, and it grows with the number of joins.

To quantify this trade-off, we group test queries based on their join count and measure both planning and execution times. Table 3 reports statistics across different groups, including the 50th–99th percentile planning latency with Dycard, as well as average planning and execution times with and without Dycard.

We observe that planning overhead increases significantly as join count rises. For example, the average planning time rises from 0.7ms (PostgreSQL) to 7.4ms (Dycard) for 2-way joins, and from 6.5ms to over 500ms for 7-way joins. However, the execution time savings are substantial—often an order of magnitude larger than the planning cost. For 5-way joins, Dycard reduces the average execution time by over 16 seconds, while incurring only 148ms of additional planning latency. This indicates that Dycard’s planning overhead is a worthwhile investment, as it enables the generation of significantly better execution plans.

To further reduce this cost, we envision several future optimizations. One approach is to use a lightweight classifier to predict whether a subquery is likely to have a large estimation error, applying Dycard’s refinement only in such cases. Another direction is to adjust the plan enumeration strategy to reduce the number of subqueries requiring correction. These ideas offer promising directions for reducing planning time with minimal impact on accuracy.

Beyond inference latency, it is also essential to evaluate whether Dycard introduces significant system-level resource overhead that could interfere with query serving. To this end, we monitor CPU and memory usage over a one-hour stable run of PostgreSQL, with Dycard enabled and disabled, respectively. The results are summarized in Table 4. We observe that Dycard incurs only a modest increase in average CPU and memory usage. While the maximum CPU utilization reaches higher spikes due to intermittent model training and inference, the 90th to 99th percentile values remain within reasonable bounds, and the system exhibits no signs of instability. Memory usage remains largely stable across both configurations, with Dycard adding less than 0.8GB on average.

These results indicate that Dycard is lightweight and practical for deployment in production environments. This efficiency is largely attributed to our streaming training design, which avoids the high overhead typically associated with centralized training and model updates. Furthermore, the system supports flexible configuration—such as reducing batch sizes to limit peak usage—or even separating Dycard and PostgreSQL onto different physical nodes if the communication overhead is acceptable.

6 Related Work

Recent years have witnessed a surge of interest in applying machine learning techniques to database query optimization [5, 16, 20, 51, 68, 73]. A wide range of studies have focused on improving query performance by enhancing various components of the traditional query optimizer, including cardinality estimation [14, 33, 44, 47, 59], cost modeling [30, 34, 61], and join ordering [9, 19, 37, 62, 67, 70].

Among these directions, cardinality estimation has attracted attention, as inaccurate estimates often lead to suboptimal plans. The goal is to capture correlations between attributes and tables that traditional estimators—relying on independence and uniformity assumptions—fail to model. Query-driven methods [22, 40, 42, 54] address this by learning a mapping from query features to output cardinalities, using supervised regression models trained on historical workloads. In contrast, data-driven approaches [12, 18, 55, 60, 64, 75] attempt to learn the joint data distribution directly, treating queries as statistical access operations over the learned distribution. Several studies have provided systematic comparisons and analyses of such methods [8, 21, 56], and benchmark datasets such as JOB[26] and STATS-CEB [15] have been proposed to reflect real-world estimation challenges.

Researchers have explored the possibility of building end-to-end learned query optimizers [35, 36, 63, 72], which directly perform the entire optimization process rather than optimizing individual components. Beyond single query optimization, some work [10, 29, 71] also considers multi-query settings and concurrency.

In addressing the challenge of training data collection, Ventura et al. [53] proposed a data-driven approach that generates large-scale query workloads along with corresponding labels. Building upon this, follow-up work [57] incorporated user interaction through a graphical interface, enabling manual intervention in query generation and labeling. The Boot framework [31] takes a different direction by modifying query execution semantics to generate training data without computing exact results, significantly improving the efficiency of sample collection in learning-based settings.

A key challenge in this area is system integration. While many learned techniques demonstrate strong offline performance, their deployment into real-world DBMSs remains difficult. Bao [35] and Lero [72] adopt external learning loops that guide plan selection over time based on feedback from actual executions. AutoSteer [1] and LEON [6] further extend these approaches with their own enhancements. PilotScope [74] proposes a middleware framework that aims to simplify the integration of learning-based components with existing DBMSs, offering modularity and runtime flexibility.

In addition, with the rapid progress of large language models in recent years, several studies [50, 65] have explored their potential applications in the context of query optimization.

7 CONCLUSIONS

We present Dycard, a lightweight and deployable refinement system designed to assist rather than replace traditional cardinality estimation in database optimizers. Unlike many existing learned methods, Dycard requires no pretraining and is capable of progressive refinement based on query feedback during normal workload execution. It introduces a novel attention-based encoding that not only improves estimation quality but also offers an interpretable view into join-crossing correlations, enhancing transparency in optimization decisions. With its low overhead and seamless integration into PostgreSQL, Dycard demonstrates strong potential for real-world deployment in dynamic query environments. Its design ensures minimal disruption to existing systems while providing consistent accuracy gains, making it a practical and robust choice for modern query optimization pipelines.

References

- [1] Christoph Anneser, Nesime Tatbul, David Cohen, Zhenggang Xu, Nikolay Pavlovich Laptev, Ryan Marcus, Prithvi Pandian, and Ryan Marcus. AutoSteer. 2023. AutoSteer: Learned Query Optimization for Any SQL Database. *Proc. VLDB Endow.* 16 (2023), 3515–3527. <https://api.semanticscholar.org/CorpusID:261259493>
- [2] Nicolò Cesa-Bianchi and Gábor Lugosi. 2006. Prediction, learning, and games. <https://api.semanticscholar.org/CorpusID:28709420>
- [3] Swati V. Chande and Madhavi Sinha. 2011. Genetic optimization for the join ordering problem of database queries. In *2011 Annual IEEE India Conference*. 1–5. <https://doi.org/10.1109/INDCON.2011.6139336>
- [4] Tianyi Chen, Jun Gao, Hedui Chen, and Yaofeng Tu. 2023. LOGER: A Learned Optimizer towards Generating Efficient and Robust Query Execution Plans. *Proc. VLDB Endow.* 16 (2023), 1777–1789. <https://api.semanticscholar.org/CorpusID:258190810>
- [5] Tianyi Chen, Jun Gao, Yaofeng Tu, and Mo Xu. 2024. GLO: Towards Generalized Learned Query Optimization. *2024 IEEE 40th International Conference on Data Engineering (ICDE) (2024)*, 4843–4855. <https://api.semanticscholar.org/CorpusID:271407402>
- [6] Xu Chen, Haitian Chen, Zibo Liang, Shuncheng Liu, Jinghong Wang, Kai Zeng, Han Su, and Kai Zheng. 2023. LEON: A New Framework for ML-Aided Query Optimization. *Proc. VLDB Endow.* 16 (2023), 2261–2273. <https://api.semanticscholar.org/CorpusID:259246460>
- [7] Xu Chen, Zhen Wang, Shuncheng Liu, Yaliang Li, Kai Zeng, Bolin Ding, Jingren Zhou, Han Su, and Kai Zheng. 2023. BASE: Bridging the Gap between Cost and Latency for Query Optimization. *Proc. VLDB Endow.* 16 (2023), 1958–1966. <https://api.semanticscholar.org/CorpusID:258718576>
- [8] Yannis Chronis, Yawen Wang, Yu Gan, Sami Abu-El-Haija, Chelsea Lin, Carsten Binnig, and Fatma Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. *ArXiv abs/2408.16170 (2024)*. <https://api.semanticscholar.org/CorpusID:272146066>
- [9] Yaohui Chu, Yizhe Liu, Yue Zhang, Xuan Hou, Longfei Yu, and Zhaohui Peng. 2024. TESSM: Tree-based Selective State Space Models for Efficient Join Order Selection Learning. *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (2024)*. <https://api.semanticscholar.org/CorpusID:273497107>
- [10] Jennie Duggan, Ugur Çetintemel, Olga Papaemmanouil, and Eli Upfal. 2011. Performance prediction for concurrent database workloads. In *ACM SIGMOD Conference*. <https://api.semanticscholar.org/CorpusID:15677153>
- [11] Yoav Freund and Robert E. Schapire. 1997. A decision-theoretic generalization of on-line learning and an application to boosting. In *European Conference on Computational Learning Theory*. <https://api.semanticscholar.org/CorpusID:6644398>
- [12] Lise Getoor, Ben Taskar, and Daphne Koller. 2001. Selectivity estimation using probabilistic models. In *ACM SIGMOD Conference*. <https://api.semanticscholar.org/CorpusID:3053207>
- [13] Goetz Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data(base) Engineering Bulletin* 18 (1995), 19–29. <https://api.semanticscholar.org/CorpusID:260706023>
- [14] Yuxing Han, Haoyu Wang, Lixiang Chen, Yifeng Dong, Xing Chen, Benquan Yu, Chengcheng Yang, and Weining Qian. 2024. ByteCard: Enhancing ByteDance’s Data Warehouse with Learned Cardinality Estimation. *Companion of the 2024 International Conference on Management of Data (2024)*. <https://api.semanticscholar.org/CorpusID:268680884>
- [15] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, G. Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *Proc. VLDB Endow.* 15 (2021), 752–765. <https://api.semanticscholar.org/CorpusID:237490370>
- [16] Roman Heinrich, Manisha Luthra, Johannes Wehrstein, Harald Kornmayer, and Carsten Binnig. 2025. How Good are Learned Cost Models, Really? Insights from Query Optimization Tasks. *ArXiv abs/2502.01229 (2025)*. <https://api.semanticscholar.org/CorpusID:276095149>
- [17] Benjamin Hilprecht and Carsten Binnig. 2022. Zero-Shot Cost Models for Out-of-the-box Learned Cost Prediction. *ArXiv abs/2201.00561 (2022)*. <https://api.semanticscholar.org/CorpusID:245650352>
- [18] Benjamin Hilprecht, Andreas Schmidt, Moritz Kulessa, Alejandro Molina, Kristian Kersting, and Carsten Binnig. 2020. DeepDB: Learn from Data, not from Queries! *VLDB* 13, 7 (2020), 992–1005.
- [19] Yesdaulet Izenov, Asoke Datta, Brian Tsan, and Florin Rusu. 2024. Sub-optimal Join Order Identification with L1-error. *Proceedings of the ACM on Management of Data* 2 (2024), 1 – 24. <https://api.semanticscholar.org/CorpusID:268765512>
- [20] Amin Kamali, Verena Kantere, and Calisto Zuzarte. 2024. Robust Query Optimization in the Era of Machine Learning: State-of-the-Art and Future Directions. *2024 IEEE 40th International Conference on Data Engineering (ICDE) (2024)*, 5371–5375. <https://api.semanticscholar.org/CorpusID:271406467>
- [21] Kyoungmin Kim, Jisung Jung, In Seo, Wook-Shin Han, Kangwoo Choi, and Jaehyok Chong. 2022. Learned Cardinality Estimation: An In-depth Study. *Proceedings of the 2022 International Conference on Management of Data (2022)*. <https://api.semanticscholar.org/CorpusID:249579044>
- [22] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter A. Boncz, and Alfons Kemper. 2018. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. *ArXiv abs/1809.00677 (2018)*. <https://api.semanticscholar.org/CorpusID:52154172>
- [23] Donald H. Kraft, Frederick E. Petry, Bill P. Buckles, and Thyagarajan Sadasivan. 1997. GENETIC ALGORITHMS FOR QUERY OPTIMIZATION IN INFORMATION RETRIEVAL: RELEVANCE FEEDBACK. <https://api.semanticscholar.org/CorpusID:59790074>
- [24] Sangoh Lee, Kyoungmin Kim, and Wook-Shin Han. 2024. ASM in Action: Fast and Practical Learned Cardinality Estimation. *Companion of the 2024 International Conference on Management of Data (2024)*. <https://api.semanticscholar.org/CorpusID:269987848>
- [25] Claude Lehmann, Pavel Sulimov, and Kurt Stockinger. 2023. Is Your Learned Query Optimizer Behaving As You Expect? A Machine Learning Perspective. *Proc. VLDB Endow.* 17 (2023), 1565–1577. <https://api.semanticscholar.org/CorpusID:261531293>
- [26] Viktor Leis, Andrey Gubichev, Atanas Mirchev, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2015. How Good Are Query Optimizers, Really? *Proc. VLDB Endow.* 9 (2015), 204–215. <https://api.semanticscholar.org/CorpusID:7953847>
- [27] Pengfei Li, Wenqing Wei, Rong Zhu, Bolin Ding, Jingren Zhou, and Hua Lu. 2023. ALECE: An Attention-based Learned Cardinality Estimator for SPJ Queries on Dynamic Workloads. *Proc. VLDB Endow.* 17 (2023), 197–210. <https://api.semanticscholar.org/CorpusID:263831415>
- [28] Rui Li, Kangfei Zhao, Jeffrey Xu Yu, and Guoren Wang. 2024. CardOOD: Robust Query-driven Cardinality Estimation under Out-of-Distribution. *ArXiv abs/2412.05864 (2024)*. <https://api.semanticscholar.org/CorpusID:274597074>
- [29] Yan Li, Liwei Wang, Sheng Wang, Yuan Sun, and Zhiyong Peng. 2022. A Resource-Aware Deep Cost Model for Big Data Query Processing. *2022 IEEE 38th International Conference on Data Engineering (ICDE) (2022)*, 885–897. <https://api.semanticscholar.org/CorpusID:251293463>
- [30] Zibo Liang, Xu Chen, Yuyang Xia, Runfan Ye, Haitian Chen, Jiandong Xie, and Kai Zheng. 2024. DACE: A Database-Agnostic Cost Estimator. *2024 IEEE 40th International Conference on Data Engineering (ICDE) (2024)*, 4925–4937. <https://api.semanticscholar.org/CorpusID:271405614>
- [31] Wan Shen Lim, Lin Ma, William Zhang, Matthew Butrovich, Samuel Arch, and Andrew Pavlo. 2024. Hit the Gym: Accelerating Query Execution to Efficiently Bootstrap Behavior Models for Self-Driving Database Management Systems. *Proc. VLDB Endow.* 17 (2024), 3680–3693. <https://api.semanticscholar.org/CorpusID:272274927>
- [32] Longxin Lin. 1992. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning* 8 (1992), 293–321. <https://api.semanticscholar.org/CorpusID:3248358>
- [33] Jie Liu, Wenqian Dong, Dong Li, and Qingqing Zhou. 2021. Fauce: Fast and Accurate Deep Ensembles with Uncertainty for Cardinality Estimation. *Proc. VLDB Endow.* 14 (2021), 1950–1963. <https://api.semanticscholar.org/CorpusID:236995517>
- [34] Shuncheng Liu, Xu Chen, Yan Zhao, Jin Chen, Ruibo Zhou, and Kai Zheng. 2022. Efficient Learning with Pseudo Labels for Query Cost Estimation. *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (2022)*. <https://api.semanticscholar.org/CorpusID:252904833>
- [35] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2020. Bao: Making Learned Query Optimization Practical. *Proceedings of the 2021 International Conference on Management of Data (2020)*. <https://api.semanticscholar.org/CorpusID:235474469>
- [36] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Chi Zhang, Mohammad Alizadeh, Tim Kraska, Olga Papaemmanouil, and Nesime Tatbul. 2019. Neo: A Learned Query Optimizer. *Proc. VLDB Endow.* 12 (2019), 1705–1718. <https://api.semanticscholar.org/CorpusID:102352007>
- [37] Ryan Marcus and Olga Papaemmanouil. 2018. Deep Reinforcement Learning for Join Order Enumeration. *Proceedings of the First International Workshop on Exploiting Artificial Intelligence Techniques for Data Management (2018)*. <https://api.semanticscholar.org/CorpusID:3854187>
- [38] Ryan Marcus and Olga Papaemmanouil. 2019. Plan-Structured Deep Neural Network Models for Query Performance Prediction. *ArXiv abs/1902.00132 (2019)*. <https://api.semanticscholar.org/CorpusID:59553521>
- [39] Guido Moerkotte, Thomas Neumann, and Gabriele Steidl. 2009. Preventing Bad Plans by Bounding the Impact of Cardinality Estimation Errors. *Proc. VLDB Endow.* 2 (2009), 982–993. <https://api.semanticscholar.org/CorpusID:7132856>
- [40] Parimarjan Negi, Ryan Marcus, Andreas Kipf, Hongzi Mao, Nesime Tatbul, Tim Kraska, and Mohammad Alizadeh. 2021. Flow-Loss: Learning Cardinality Estimates That Matter. *Proc. VLDB Endow.* 14 (2021), 2019–2032. <https://api.semanticscholar.org/CorpusID:231592367>

- [41] Parimarjan Negi, Ryan Marcus, Hongzi Mao, Nesime Tatbul, Tim Kraska, and Mohammad Alizadeh. 2020. Cost-Guided Cardinality Estimation: Focus Where it Matters. *2020 IEEE 36th International Conference on Data Engineering Workshops (ICDEW)* (2020), 154–157. <https://api.semanticscholar.org/CorpusID:218676487>
- [42] Parimarjan Negi, Ziniu Wu, Andreas Kipf, Nesime Tatbul, Ryan Marcus, Samuel Madden, Tim Kraska, and Mohammad Alizadeh. 2023. Robust Query Driven Cardinality Estimation under Changing Workloads. *Proc. VLDB Endow.* 16 (2023), 1520–1533. <https://api.semanticscholar.org/CorpusID:258190940>
- [43] Jennifer Ortiz, Magdalena Balazinska, Johannes Gehrke, and S. Sathya Keerthi. 2019. An Empirical Analysis of Deep Learning for Cardinality Estimation. *ArXiv abs/1905.06425* (2019). <https://api.semanticscholar.org/CorpusID:155100270>
- [44] Silvan Reiner and Michael Grossniklaus. 2023. Sample-Efficient Cardinality Estimation Using Geometric Deep Learning. *Proc. VLDB Endow.* 17 (2023), 740–752. <https://api.semanticscholar.org/CorpusID:268312267>
- [45] Patricia G. Selinger, Morton M. Astrahan, Donald D. Chamberlin, Raymond A. Lorie, and Thomas G. Price. 1979. Access path selection in a relational database management system. In *ACM SIGMOD Conference*. <https://api.semanticscholar.org/CorpusID:8537523>
- [46] Tarique Siddiqui, Alekh Jindal, Shi Qiao, Hiren Patel, and Wangchao Le. 2020. Cost Models for Big Data Query Processing: Learning, Retrofitting, and Our Findings. *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (2020). <https://api.semanticscholar.org/CorpusID:211572502>
- [47] Xun Song, Jiaqi Zheng, Hao Qian, Shiju Zhao, Hongxuan Zhang, Xuntao Pan, and Guihai Chen. 2023. Couper: Memory-Efficient Cardinality Estimation under Unbalanced Distribution. *2023 IEEE 39th International Conference on Data Engineering (ICDE)* (2023), 2753–2765. <https://api.semanticscholar.org/CorpusID:260171626>
- [48] Michael Stillger, Guy M. Lohman, Volker Markl, and Mokhtar Kandil. 2001. LEO - DB2's LEarning Optimizer. In *Very Large Data Bases Conference*. <https://api.semanticscholar.org/CorpusID:4836343>
- [49] Ji Sun and Guoliang Li. 2019. An End-to-End Learning-based Cost Estimator. *Proc. VLDB Endow.* 13 (2019), 307–319. <https://api.semanticscholar.org/CorpusID:174802790>
- [50] Jie Tan, Kangfei Zhao, Rui Li, Jeffrey Xu Yu, Chengzhi Piao, Hong Cheng, Helen M. Meng, Deli Zhao, and Yu Rong. 2025. Can Large Language Models Be Query Optimizer for Relational Databases? *ArXiv abs/2502.05562* (2025). <https://api.semanticscholar.org/CorpusID:276249587>
- [51] Jeffrey Tao, Natalie Maus, Haydn Thomas Jones, Yimeng Zeng, Jacob R. Gardner, and Ryan Marcus. 2025. Learned Offline Query Planning via Bayesian Optimization. *ArXiv abs/2502.05256* (2025). <https://api.semanticscholar.org/CorpusID:276249530>
- [52] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Neural Information Processing Systems*. <https://api.semanticscholar.org/CorpusID:13756489>
- [53] Francesco Ventura, Zoi Kaoudi, Jorge-Arnulfo Quiané-Ruiz, and Volker Markl. 2021. Expand your Training Limits! Generating Training Data for ML-based Data Management. *Proceedings of the 2021 International Conference on Management of Data* (2021). <https://api.semanticscholar.org/CorpusID:235473953>
- [54] Fang Wang, Xiao Yan, Man Lung Yiu, Li Shuai, Zunyao Mao, and Bo Tang. 2023. Speeding Up End-to-end Query Execution via Learning-based Progressive Cardinality Estimation. *Proceedings of the ACM on Management of Data* 1 (2023), 1 – 25. <https://api.semanticscholar.org/CorpusID:259077235>
- [55] Jiayi Wang, Chengliang Chai, Jiabin Liu, and Guoliang Li. 2021. FACE: A Normalizing Flow based Cardinality Estimator. *Proc. VLDB Endow.* 15 (2021), 72–84. <https://api.semanticscholar.org/CorpusID:244771794>
- [56] Xiaoying Wang, Changbo Qu, Weiyuan Wu, Jiannan Wang, and Qingqing Zhou. 2020. Are We Ready For Learned Cardinality Estimation? *ArXiv abs/2012.06743* (2020). <https://api.semanticscholar.org/CorpusID:229157499>
- [57] Robin Van De Water, Francesco Ventura, Zoi Kaoudi, Jorge-Arnulfo Quiané-Ruiz, and Volker Markl. 2022. Farming Your ML-based Query Optimizer's Food. *2022 IEEE 38th International Conference on Data Engineering (ICDE)* (2022), 3186–3189. <https://api.semanticscholar.org/CorpusID:251304055>
- [58] Lucas Woltmann, Claudio Hartmann, Maik Thiele, Dirk Habich, and Wolfgang Lehner. 2019. Cardinality estimation with local deep learning models. *Proceedings of the Second International Workshop on Exploiting Artificial Intelligence Techniques for Data Management* (2019). <https://api.semanticscholar.org/CorpusID:167210229>
- [59] Ziniu Wu, Parimarjan Negi, Mohammad Alizadeh, Tim Kraska, and Sam Madden. 2022. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1 (2022), 1 – 27. <https://api.semanticscholar.org/CorpusID:254564345>
- [60] Ziniu Wu and Amir Shaikhha. 2020. BayesCard: A Unified Bayesian Framework for Cardinality Estimation. *ArXiv abs/2012.14743* (2020). <https://api.semanticscholar.org/CorpusID:229923940>
- [61] Yu Yan, Hongzhi Wang, Junfang Huang, Dake Zhong, Man Yang, Kaixin Zhang, Tao Yu, and Tianqing Wang. 2023. QCFE: An Efficient Feature Engineering for Query Cost Estimation. *2024 IEEE 40th International Conference on Data Engineering (ICDE)* (2023), 4302–4315. <https://api.semanticscholar.org/CorpusID:263605699>
- [62] Zhengtong Yan, Valter Uotila, and Jiaheng Lu. 2023. Join Order Selection with Deep Reinforcement Learning: Fundamentals, Techniques, and Challenges. *Proc. VLDB Endow.* 16 (2023), 3882–3885. <https://api.semanticscholar.org/CorpusID:261663211>
- [63] Zongheng Yang, Wei-Lin Chiang, Sifei Luan, Gautam Mittal, Michael Luo, and Ion Stoica. 2022. Balsa: Learning a Query Optimizer Without Expert Demonstrations. *Proceedings of the 2022 International Conference on Management of Data* (2022). <https://api.semanticscholar.org/CorpusID:245704291>
- [64] Zongheng Yang, Eric Liang, Amog Kamsetty, Chenggang Wu, Yan Duan, Peter Chen, P. Abbeel, Joseph M. Hellerstein, Sanjay Krishnan, and Ion Stoica. 2019. Deep Unsupervised Cardinality Estimation. *Proc. VLDB Endow.* 13 (2019), 279–292. <https://api.semanticscholar.org/CorpusID:208202391>
- [65] Zhiming Yao, Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. 2025. A Query Optimization Method Utilizing Large Language Models. *ArXiv abs/2503.06902* (2025). <https://api.semanticscholar.org/CorpusID:276903444>
- [66] Xiang Yu. 2022. Cost-based or Learning-based? A Hybrid Query Optimizer for Query Plan Selection. *Proc. VLDB Endow.* 15 (2022), 3924–3936. <https://api.semanticscholar.org/CorpusID:253519234>
- [67] Xiang Yu, Guoliang Li, Chengliang Chai, and Nan Tang. 2020. Reinforcement Learning with Tree-LSTM for Join Order Selection. *2020 IEEE 36th International Conference on Data Engineering (ICDE)* (2020), 1297–1308. <https://api.semanticscholar.org/CorpusID:218906511>
- [68] Xin Zhang and Ahmed Eldawy. 2024. Spatial Query Optimization With Learning. *Proc. VLDB Endow.* 17 (2024), 4245–4248. <https://api.semanticscholar.org/CorpusID:272728000>
- [69] Kangfei Zhao, Jeffrey Xu Yu, Zongyan He, Rui Li, and Hao Zhang. 2022. Lightweight and Accurate Cardinality Estimation by Neural Network Gaussian Process. *Proceedings of the 2022 International Conference on Management of Data* (2022). <https://api.semanticscholar.org/CorpusID:249578927>
- [70] Weiqi Zhou, Siyu Zhan, Bo Dai, and Lei Guo. 2022. SOAR: A Learned Join Order Selector with Graph Attention Mechanism. *2022 International Joint Conference on Neural Networks (IJCNN)* (2022), 1–8. <https://api.semanticscholar.org/CorpusID:252625627>
- [71] Xuanhe Zhou, Ji Sun, Guoliang Li, and Jianhua Feng. 2020. Query performance prediction for concurrent queries using graph embedding. *Proceedings of the VLDB Endowment* 13 (2020), 1416 – 1428. <https://api.semanticscholar.org/CorpusID:219557876>
- [72] Rong Zhu, Wei Chen, Bolin Ding, Xingguang Chen, Andreas Pfadler, Ziniu Wu, and Jingren Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *Proc. VLDB Endow.* 16 (2023), 1466–1479. <https://api.semanticscholar.org/CorpusID:256846635>
- [73] Rong Zhu, Lianggui Weng, Bolin Ding, and Jingren Zhou. 2024. Learned Query Optimizer: What is New and What is Next. *Companion of the 2024 International Conference on Management of Data* (2024). <https://api.semanticscholar.org/CorpusID:269987252>
- [74] Rong Zhu, Lianggui Weng, Wenqing Wei, Di Wu, Jiazhen Peng, Yifan Wang, Bolin Ding, Defu Lian, Bolong Zheng, and Jingren Zhou. 2024. PilotScope: Steering Databases with Machine Learning Drivers. *Proc. VLDB Endow.* 17 (2024), 980–993. <https://api.semanticscholar.org/CorpusID:268724503>
- [75] Rong Zhu, Ziniu Wu, Yuxing Han, Kai Zeng, Andreas Pfadler, Zhengping Qian, Jingren Zhou, and Bin Cui. 2020. FLAT: Fast, Lightweight and Accurate Method for Cardinality Estimation. *ArXiv abs/2011.09022* (2020). <https://api.semanticscholar.org/CorpusID:227012579>